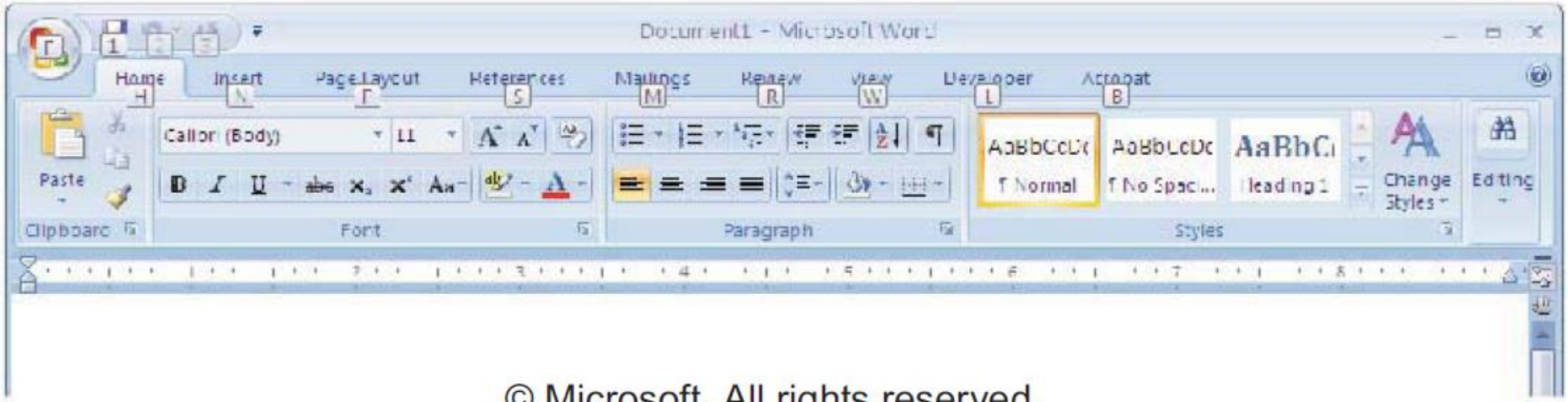
Layout



© Microsoft. All rights reserved.

ribbon, introduced in Microsoft Office 2007

menubar and toolbars into a single common widget

strict **external consistency** has been thrown out the window – Office no longer has a menubar or toolbar

task analysis: they signed up 1000s of Office users to a program that collected statistics on how frequently they used Office commands and in which order – huge amounts of data that directly drove how commands were grouped into the command tabs, and which commands appear on command tabs as opposed to being buried in deeper dialogs

How about web pages? server logs

more **feedback** about what a command will do, by showing a preview of its effect right in the document while you're mousing over the command

- As long as your computer is fast enough to do it within **100ms**, a tremendous improvement to the visibility and feedback of the interface

Today's Topics

- CSS
- Automatic layout
- Constraints

Cascading Style Sheets (CSS)

- Key idea: separate the **structure** of the UI (view tree) from details of **presentation**
  - HTML is structure, CSS is presentation
- Two ways to use CSS
  - As an attribute of a particular HTML element

```
<button style="font-weight:bold;"> Cut </button>
```

  - As a style sheet defining style rules for many HTML elements at once

```
<style>
   button { font-weight:bold; }
</style>
```

CSS Selectors

- Each rule in a style sheet has a **selector** pattern that matches a set of HTML elements

Tag name

```
button { font-weight:bold; }
```

ID

```
#main { background-color:
        rgb(100%,100%,100%); }
```

Class attribute

```
.toolbarButton { font-size: 12pt; }
```

Element paths

```
#toolbar button { display: hidden; }
```

```
<div id="main">
  <div id="toolbar">
    <button class="toolbarButton">
      <img src="cut.png"></img>
    </button>
  </div>
  <textarea id="editor"></textarea>
</div>
```

Selectors are also used by jQuery to select and operate on nodes in the DOM tree

Cascading and Inheritance

- If multiple rules apply to the same element, rules are automatically combined with **cascading** precedence
  - Source: browser defaults < web page < user overrides

    Browser says: a { text-decoration: underline; }

    Web page says: a { text-decoration: none; }

    User says: a { text-decoration: underline; }

  - Rule specificity: general selectors < specific selectors

    button { font-size: 12pt; }

    .toolbarButton { font-size: 14pt; }

- Styles can also be **inherited** from element's parent
  - This is the default for simple styles like font, color, and text properties

    body { font-size: 12pt; }

difficulties of debugging declarative specifications: You may add a rule to the style sheet, maybe trying to change a button's font size, only to see *no change* in the result because some other rule that you aren't aware of is taking precedence

Declarative Styles vs. Procedural Styles

CSS

```
// found in a <style> element
button { font-size: 12pt; font-weight: bold; }
```

jQuery

```
// in a <script> element
$("button").css("font-size", "12pt").css("font-weight", "bold");
```

Automatic Layout

- **Layout** determines the sizes and positions of components on the screen
  - Also called geometry in some toolkits
- Declarative layout
  - CSS styles
- Procedural layout
  - Write Javascript code to compute positions and sizes

Reasons to Do Automatic Layout

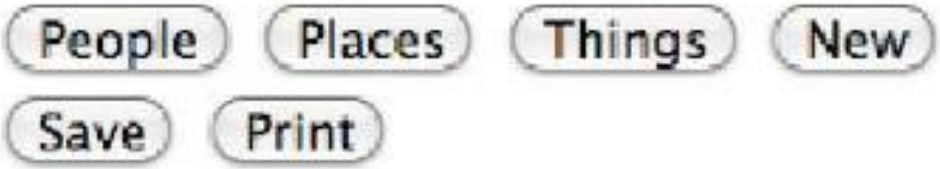
- Higher level programming
  - Shorter, simpler code
- Adapts to change
  - Window size
  - Font size
  - Widget set (or theme or skin)
  - Labels (internationalization)
  - Adding or removing nodes

Flow Layout

- Left-to-right, automatically-wrapping
- CSS calls this “inline” layout

  display: inline
- Many elements use inline layout by default

```
<button>People</button>
<button>Places</button>
<button>Things</button>
<button>New</button>
<button>Save</button>
<button>Print</button>
```



Block Layout

- Blocks are laid out vertically
  - display: block
  - divs default to block layout
- Inline blocks are laid out in flow
  - display: inline-block

```
<div>
<button>People</button>
<button>Places</button>
<button>Things</button>
</div>
<div>
<button>New</button>
<button>Save</button>
<button>Print</button>
</div>
```

Float Layout

- Float pushes a block to left or right edge

```
<style>
.navbar { float: left; }
.navbar button { display: block; }
</style>

<div class="navbar">
<button>People</button>
<button>Places</button>
<button>Things</button>
</div>

<div>
<button>New</button>
<button>Save</button>
<button>Print</button>
</div>
```

Grid Layout

- Blocks & floats are typically not enough to enforce all the alignments you want in a UI
- Use tables instead

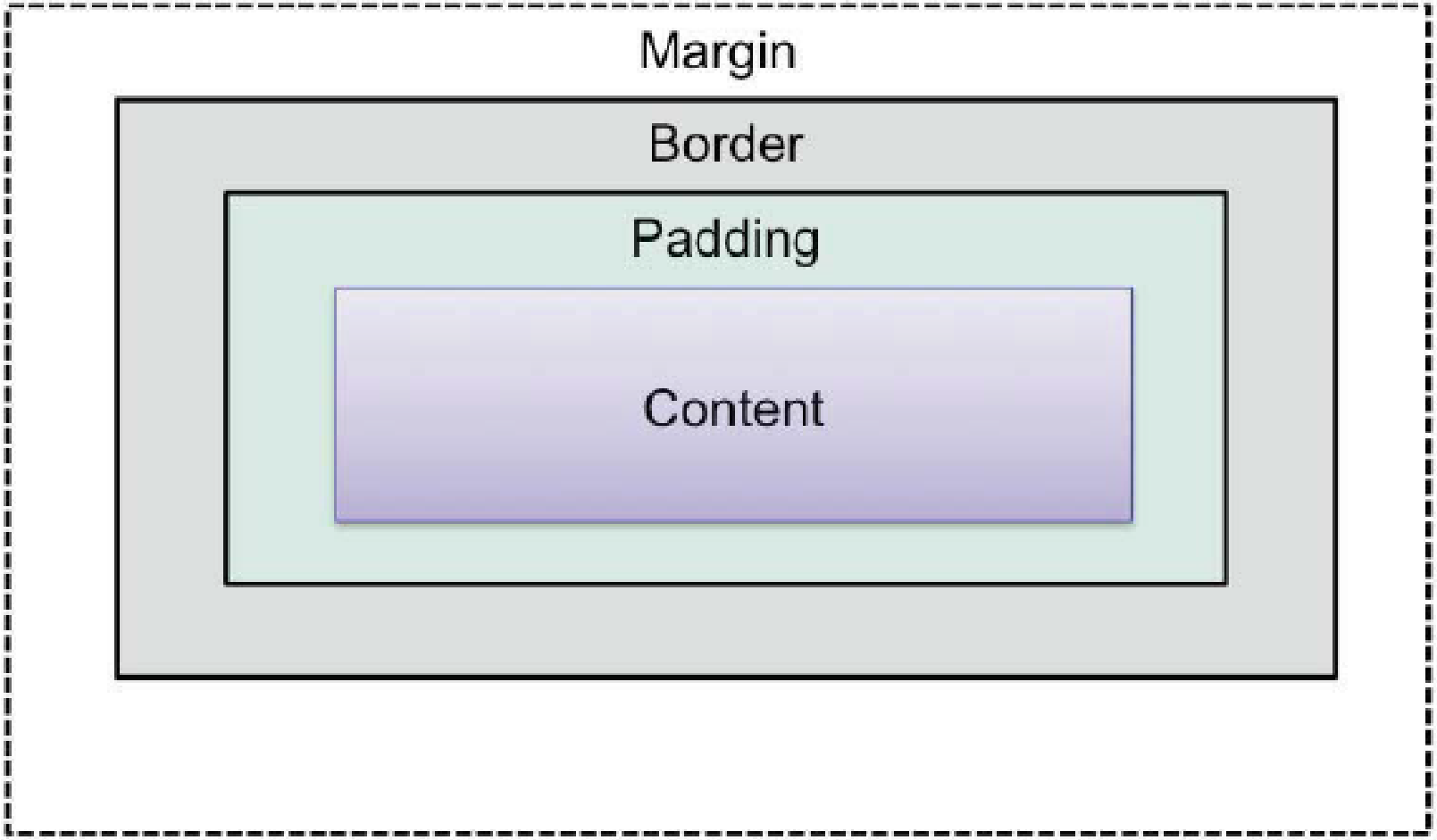
```
<table>
<tr><td>Name:</td>
      <td><input type="text" /></td>
</tr>
<tr><td>Groups:</td>
      <td><textarea></textarea></td>
</tr>
</table>
```

Name:

Groups:

- More on this with Bootstrap

Margins, Borders, & Padding



Space-Filling & Alignment

- width: 100% , height: 100% consumes all of the space available in the parent
- vertical-align moves a node up and down in its parent's box
  - baseline is good for lining up labels with textboxes
  - top and bottom are useful for other purposes
- Centering
  - margin: auto for boxes
  - text-align: center for inlines

Absolute Positioning

- Setting position & size explicitly
  - in coordinate system of entire window, or of node's parent
  - CSS has several units: px, em, ex, pt
  - mostly useful for popups

```
<style>
button { position: absolute;
         left: 5px;
         top: 5px; }
</style>
```

Print s the user interface goes here

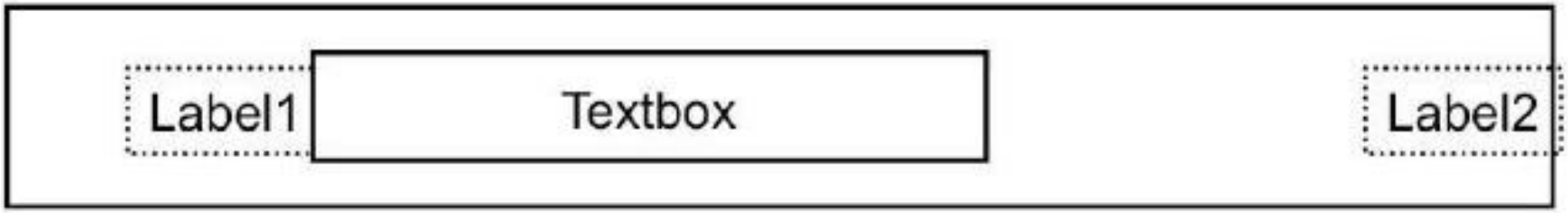
Constraints

- **Constraint** is a relationship among variables that is automatically maintained by system
  - **Constraint propagation:** When a variable changes, other variables are automatically changed to satisfy constraint

layout managers are a limited form of constraint system

Each layout manager represents a set of relationships among the positions and sizes of the children of a single container; and layout propagation finds a solution that satisfies these relationships

Using Constraints for Layout




```
label1.left = 5
label1.width = textwidth(label1.text, label1.font)
label1.right = textbox.left
label1.left + label1.width = label1.right

textbox.width >= parent.width / 2
textbox.right <= label2.left

label2.right = parent.width
```

One-way constraints are like spreadsheets – you can think of every variable like a spreadsheet cell with a formula in it calculating X=f(X1,X2,X3,…)

worry about cycles: if variable X is computed from variable Y, but variable Y must be computed from variable X, how do you compute it?

- Simply disallow cycles (spreadsheets consider them errors, for example)
- break the cycle by reusing the old (or default) value for one of the variables; so compute variable Y using X's old value, then compute a new value for X using Y

Conflicting constraints are another problem – causing the constraint system to have no solution.
can be resolved by constraint hierarchies (**like CSS**)

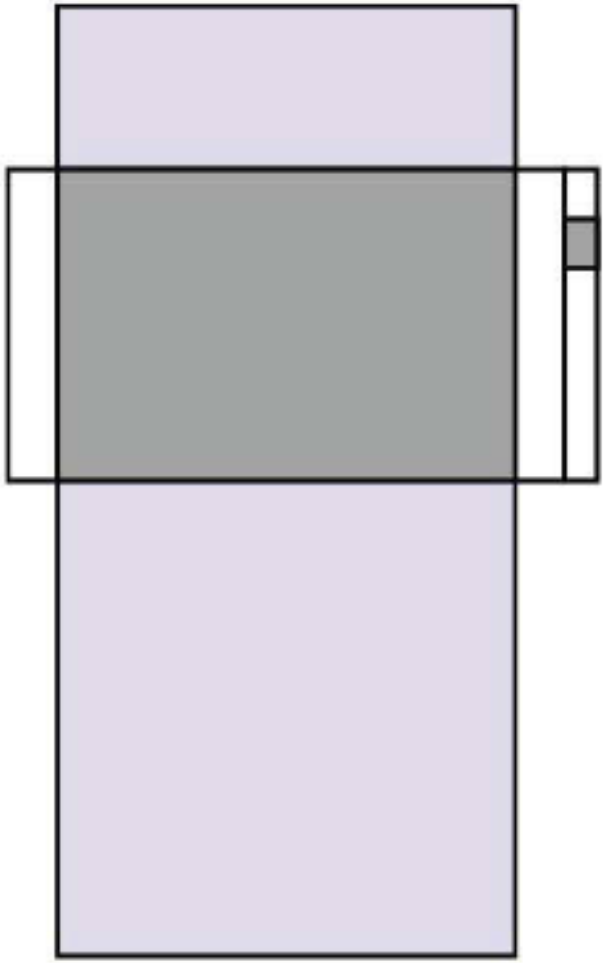
Using Constraints for Behavior

- Input
  - checker.(x,y) = mouse.(x,y)
    if mouse.button1 && mouse.(x,y) in checker
- Output
  - checker.dropShadow.visible = mouse.button1 && mouse.(x,y) in checker
- Interactions between components
  - deleteButton.enabled = (textbox.selection != null)
- Connecting view to model
  - checker.x = board.find(checker).column * 50

alternative is writing **procedural code** – an event handler that fires when one of the dependent variables changes (like mouseMoved for the Mouse position, or selectionChanged for the textbox selection)

Constraints Are Declarative UI

$$\frac{\text{scrollbar.thumb.y}}{\text{scrollbar.track.height} - \text{scrollbar.thumb.height}} = \frac{\text{-scrollpane.child.y}}{\text{scrollpane.child.height} - \text{scrollpane.height}}$$



you can *think* about your user interface in terms of constraints, and *document your code* that way

it’s easier to generate procedural code once you’ve clearly stated **what** you want (declaratively)

If you state a constraint equation, then you know which events you have to listen for (any changes to the variables in your equation), and you know what those event handlers should do (solve for the other variables in the equation)

Writing procedural code for the scrollpane is much easier if you’ve already written the constraint relationship

Summary

- Automatic layout adapts to different platforms and UI changes
- CSS for layout
- Constraints are useful for more than just layout